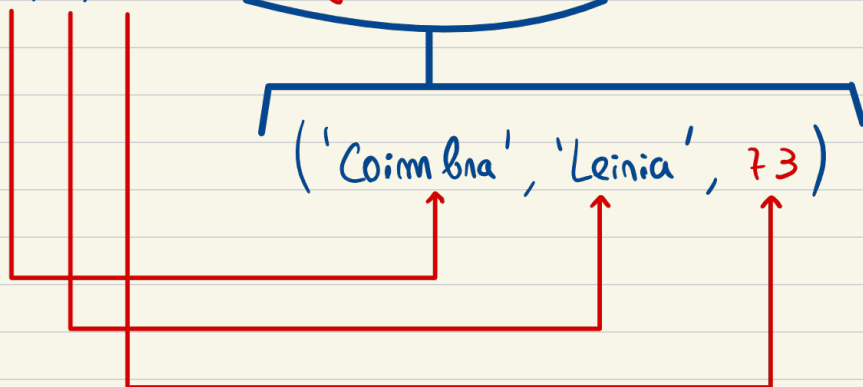


ex 7

for A, B, D in self.connections



Se A == C1 & B == C2 ou vice-versa então

retorna o custo (D)

ex 8

```

98 # procurar a solucao
99 def search(self, limit=None):
100     while self.open_nodes != []:
101         node = self.open_nodes.pop(0)
102         if self.problem.goal_test(node.state):
103             self.solution = node
104             self.length = node.depth
105             self.terminals = len(self.open_nodes) + 1 # nós que ficaram por expandir em open_nodes são folhas
106             return self.get_path(node)
107         self.non_terminals += 1 # não é solução, logo está a ser expandido
108         lnewnodes = []
109         for a in self.problem.domain.actions(node.state):
110             newstate = self.problem.domain.result(node.state, a)
111             if newstate not in self.get_path(node):
112                 new_cost = node.cost + self.problem.domain.cost(node.state, a) # custo acumulado
113                 newnode = SearchNode(newstate, node, new_cost)
114                 if limit is None or newnode.depth <= limit:
115                     lnewnodes.append(newnode)
116                 self.add_to_open(lnewnodes)
117         return None
  
```

Domínio de procura (instância da classe 'Cidades')

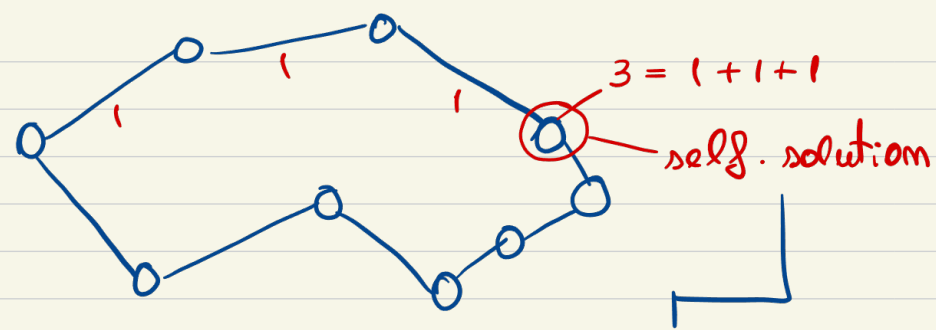
Chama o método cost() no domínio 'Cidades'

estado atual

Ação a realizar (tuplo)

(cidade_origem, cidade_destino)

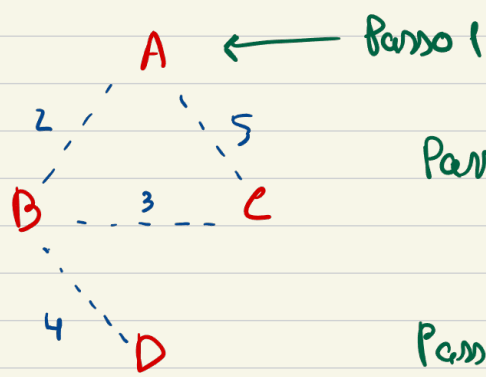
ex 9



c/ o código anterior ele guarda o valor acumulado

Só precisamos de indexar o custo a solução (se for encontrada)

Pesquisa uniforme $A \rightsquigarrow C$



Passo 2 - expande A

- B(2)
- C(5)

Passo 3 - expande B

- C(5)
- D(6)

4 + 2

```
elif self.strategy == 'uniform':  
    self.open_nodes.extend(lnewnodes)  
    self.open_nodes.sort(key=lambda node: node.cost)  
    pass
```

Ordeman do menor custo para o maior,
depois no código inicial, ele vai testar esse - custo e vai verificar.